

QK2 Recovery Document (A1)

A1 is an **ordinary printed webpage** — a recipe, a guitar tab, travel advice — whose useful body is the cloak and whose **footer resembles the page's source URL** while carrying **authenticated ciphertext of the 256-bit Seed-A entropy**. Encryption is MANDATORY. It can sit visibly among ordinary papers; there is no naked seed phrase, no conspicuous token, and no crypto label anywhere on the sheet. The AEAD capsule crypto is now **frozen — ChaCha20-Poly1305 (RFC 8439)** keyed by an HKDF-SHA256 document key; the RS/base32/framing print codec below is a reference construction whose **physical sizes stay unfrozen behind Gate A print/OCR trials**.

ARTIFACT	PAYLOAD	SEED A	DECRYPT KEY	CRYPTO	SIZES
A1 · disguised webpage	AEAD footer ciphertext	256-bit BIP-39 entropy	A2 · on B and C	ChaCha20-Poly1305 · nonce	unfrozen — recompute

PROJECT STATUS — SELECTED DIRECTION, PRE-GATE-A, NOT IMPLEMENTED

Air-gapped calculator-disguised 2-of-3 co-signer; Seed A absent at rest and restored transiently only inside the replaceable terminal. QR and microSD are **both required transports** (microSD is the lower-risk first path); no mechanical redesign authorized. Camouflage of A1 — a disguised printed webpage whose footer resembles its source URL — is a **system requirement**, not a threat-model line; stealth is a first layer of defence and NEVER a cryptographic factor or a substitute for the 2-of-3 threshold. The **coding math** below is a reference construction that round-trips in a stdlib codec (evidence level 1). The AEAD is now frozen — **ChaCha20-Poly1305 (RFC 8439)**. The **footer-in-URL disguise, OCR capture performance, card-side A2 release, and the physical print-codec sizing are all open** (Gate A). No independent decoder, disguised-webpage capture trial, or target implementation exists yet. The footer layout is NOT an interface: the RS parameters, encoded length, base32 counts, and transcription grouping are UNFROZEN and must be recomputed from Gate A print/OCR trials — the capsule crypto (ChaCha20-Poly1305, HKDF document key, stored 96-bit nonce, full-wallet id AAD, 128-bit tag) is fixed.

the capture type (capture type, recovery, and document type), and the document type, but no independent implementation or target vectors exist yet. Do not treat any of the print numbers as fixed until a real disguised-webpage capture study is run.

1. [A1 as a disguised webpage](#)
2. [Layer 1 — the AEAD capsule](#)
3. [Layer 2 — RS error correction](#)
4. [Layer 3 — base32 + framing](#)
5. [OCR reader rules](#)
6. [Versioning & fallback](#)
7. [The A2 key & camouflage](#)

01 A1 AS A DISGUISED WEBPAGE

The artifact is an ordinary printed page; its footer resembles a URL

A1 is not a "recovery sheet." It is an **ordinary printed webpage** — a recipe, a guitar tab, a page of travel advice — whose useful body is the camouflage. The recovery payload lives in the **footer**, **formatted to resemble the page's source URL** (a long path/query string), carrying authenticated ciphertext of the 256-bit Seed-A entropy. A naked BIP-39 phrase, an obvious `cv0` /token string, a crypto label, or any conspicuous "recovery block" would defeat the product and must not appear. The RS/base32/framing codec below is how those footer characters are error-corrected; its sizes are a **reference construction**, unfrozen behind Gate A physical print/OCR trials — the AEAD is now fixed (ChaCha20-Poly1305, RFC 8439), but the print sizing is not.

The illustrative pipeline: an AEAD capsule (Seed-A entropy encrypted, with a stored random nonce and identity binding) is error-corrected with Reed–Solomon over GF(256), then repacked to a base32 alphabet and framed so the footer reads like a plausible URL tail. Every size shown is produced by the

stdlib reference codec (coding layer; the AEAD is out of stdlib scope) for *one* illustrative capsule length and **must be recomputed once Gate A print/OCR trials fix the RS redundancy and footer sizing**. The capsule crypto is frozen (ChaCha20-Poly1305); the print-codec dimensions are not.

ChaCha20-Poly1305 capsule — coding-ver || crypto-ver || nonce(12) || ciphertext(32) || tag(16) , wallet_id-bound AAD **crypto frozen**

▼ **RS(n,k) over GF(2⁸): parity bytes (recompute n,k from Gate A print/OCR trials)**

RS codeword — data + parity, systematic

n bytes · recompute

▼ **base32 repack (8-bit → 5-bit)**

custom-base32 symbols — codeword bits → symbols

chars · recompute

▼ **frame as URL-like footer: path/query shape + polymod checksum**

Printed footer — resembles the page's source URL; no visible crypto marker

length · recompute

THE OLD 49 / 83 / 133 / 142 FIGURES ARE NOT NORMATIVE

Those numbers assumed a **49-byte capsule with a fixed (implicit) nonce** — an assumption now rejected. The frozen **ChaCha20-Poly1305** capsule carries a **stored random 96-bit nonce** plus identity/version binding, making it longer than the old 49-byte fixed-nonce layout and forcing a different RS(n,k) and encoded length; the whole 49/83/133/142 layout changes. Treat those figures as a historical worked example, **not a wire specification**. The capsule crypto (AEAD, HKDF document key, nonce, AAD, tag) is now frozen; the RS parameters, encoded length, base32 counts, and transcription grouping remain UNFROZEN and must be recomputed from Gate A print/OCR trials.

TERMINOLOGY — THIS IS NOT STANDARD BECH32, AND THE FOOTER MUST NOT LOOK LIKE CRYPTO

Standard BIP-173 Bech32 requires an HRP plus the 1 separator and is limited to **90 characters**. The footer has no such separator and exceeds that limit, so its encoding is a **custom base32 alphabet plus a Bech32-derived polymod checksum** — not "a Bech32 token." More important for the product: the printed result must read as a **plausible source URL** in the page footer, never as a labelled token. It still needs its own internal format name and published vectors, but that name is a codec identifier, not printed text on A1.

REFERENCE CODEC OUTPUT (CODING LAYER ONLY)

The stdlib reference codec exercises the RS/base32/checksum *coding* for a demo capsule: the polymod checksum verifies and a clean parse recovers the exact capsule bytes. It does **not** compute the AEAD (the ChaCha20-Poly1305 capsule is out of stdlib scope), and its byte counts are placeholders pending the Gate-A print sizing.

02 LAYER 1

The AEAD capsule – ChaCha20–Poly1305, encryption is MANDATORY (crypto frozen)

The footer carries an **authenticated encryption** of Seed A, encoded as its 32-byte BIP-39 *entropy* (not words). Encryption is mandatory, not a fallback: A1 alone must reveal nothing. The AEAD is **ChaCha20-Poly1305 (RFC 8439, IETF)** from a reviewed constant-time implementation — portable on the ARM target without hardware AES. The **root key is A2**, a fresh 256-bit random key generated at provisioning, duplicated onto **both** cards B and C, and wiped from the terminal; A2 is not a Bitcoin signing key. The AEAD is never keyed with A2 directly: the per-document key is **HKDF-SHA256(IKM=A2, salt=wallet_id, info="QuietKey/A1/v1")**, so each wallet gets an independent document key. The crypto layout is now frozen; only the RS/base32 print sizing built on top of it stays Gate-A-unfrozen, so those byte counts are not CI-drift-lockable yet.

```

capsule = coding-ver || crypto-ver || nonce(12) || ciphertext(32) || tag(16)    crypto layout frozen; RS/print sizing still Gate A
cipher  = ChaCha20-Poly1305 (RFC 8439, IETF AEAD)    reviewed constant-time impl; portable on ARM without hardware AES; runs in t
  root    = A2 (fresh 256-bit random, provisioned onto cards B and C)    a bearer card releases A2 over a minimal card session
  key     = HKDF-SHA256(IKM=A2, salt=wallet_id, info="QuietKey/A1/v1")    per-document key; A2 is never used as the AEAD key direct
  nonce   = fresh random 96-bit, stored in the capsule                    no fixed/implicit nonce
  AAD     = magic || coding-ver || crypto-ver || network || full wallet_id    wallet_id = SHA256(recv_desc || 0x00 || change_desc), read
  plaintext = Seed-A entropy (32 bytes)
  tag     = 128-bit Poly1305 tag
who     = the REPLACEABLE TERMINAL derives the document key and performs AEAD decryption after a bearer card (B or C) releases A
verify  = the 128-bit AEAD tag is the FINAL cryptographic authority; fail closed on any mismatch; never output unauthenticated p
after   = derive the wallet fingerprint from the recovered seed – display it, never store it; never print/export plaintext Seed

```

RANDOM NONCE IS REQUIRED

Store a **freshly random 96-bit nonce in the capsule**; do not rely on a fixed or implicit nonce. A per-wallet HKDF document key plus a fresh stored nonce keeps ChaCha20-Poly1305's nonce requirement simple, but a "unique key per wallet" argument alone is not sufficient unless reprints, migration, retries, and re-provisioning can *never* reuse the same key/nonce with a different plaintext — hard to guarantee operationally, so the nonce is always fresh and stored. Bind **magic, coding version, crypto version, network, and the full wallet_id** in the AAD so a key or capsule cannot be cross-substituted between documents. Reprinting the identical generated page reuses the identical capsule and is safe; changing Seed A or the wallet requires a fresh nonce, and a newly provisioned wallet gets a new A2 and wallet_id.

KEEP THE AEAD OFF THE CARD – SIMPLER RECOVERY MODEL

The AEAD is not coupled to whatever a particular JavaCard happens to expose: **ChaCha20-Poly1305** is a portable software AEAD run in the terminal, precisely so the document format never depends on card hardware. The model is: a bearer card (B or C) releases **A2** over a minimal card session, and the **replaceable terminal** derives the HKDF document key and decrypts. Seed B and Seed C are never exportable; A2 is exportable by design. The AEAD is frozen; independent implementation, published vectors, and the no-unauthenticated-release proof remain Gate A.

Reed–Solomon over GF(256) – a byte code, erasure–first

Reed–Solomon over GF(2⁸) (primitive polynomial 0x11D, generator $\alpha=2$, fcr 0), systematic so the leading codeword bytes *are* the capsule; parity bytes give the correction budget $2 \cdot \text{errors} + \text{erasures} \leq \text{parity}$. It is a **shortened** code (natural RS length over GF(256) is 255). The specific RS(n,k) — how many data and parity bytes — **must be recomputed from Gate A print/OCR damage trials**; the capsule crypto length is now settled by the frozen ChaCha20–Poly1305 layout, but the redundancy needed for real-world print damage is not. The old RS(83,49) held only for the rejected 49-byte fixed-nonce capsule. The exact generator polynomial $g(x) = \prod (x - \alpha^i)$, the shortening/zero-pad convention, and big-endian byte order must be published as normative vectors alongside an independent decoder before this is more than a reference construction.

WHY RS CORRECTION IS ALLOWED HERE – AND ITS PRECONDITION

BIP-173 deliberately **warns against error-correcting** Bech32, because correction erodes the code's error-*detection* guarantee. RS correction is safe only when some independent authority can reject a mis-correction, so a wrong symbol fixed to a wrong value can never produce an accepted seed. Because encryption is mandatory, that authority is the **independently keyed AEAD tag**: a mis-correction fails the tag. This dependency is load-bearing — the decoder must **never output unauthenticated plaintext**, and the AEAD tag, not the polymod checksum or RS success, decides acceptance of the footer.

Wallet-level acceptance gate. Beyond the AEAD tag, before any funds move the recovered Seed A must pass a descriptor-comparison authenticity check: derive A's origin/xpub from the restored seed and compare it against the **independently-stored descriptor** (matching fingerprint and the expected descriptor checksum), and confirm D-derived scripts match the funded UTXOs. Note a descriptor checksum is error-detection, not cryptographic authentication, so this comparison supplements — never replaces — the AEAD tag.

ERASURE ECONOMICS – THE READER'S WHOLE JOB

A confidently-wrong character (silent misread) corrupts the **one or two RS byte-symbols** its bits overlap (per the bit-to-byte mapping below). An **undetected** error — an unknown-position wrong symbol — costs **two parity units per corrupted symbol**, so a single wrong character can consume **two or four** parity units depending on whether it taints one symbol or two. A character marked as an **erasure** (known position, unknown value) costs only **one** parity unit per erased symbol. So the OCR reader must convert every uncertain read into an erasure and **never guess** — up to halving the parity cost and eliminating undetected corruption. The reference codec round-trip (coding layer only) recovers up to the RS parity budget of symbol-erasures and refuses (fail-closed) beyond it. The negative test corpus must exercise **wrong-character (silent-misread) cases, not only erasures**, so the two-or-four-parity-unit cost of an undetected error is actually measured.

Character → symbol erasure mapping

Because 5-bit characters and 8-bit RS symbols don't align, an erased character taints the one or two bytes its bits overlap. The reader marks those RS symbols as erasures:

```
erased char index c ⇒ RS symbols [5c/8] ... [(5c+4)/8] (1 or 2 bytes)
budget check on the SYMBOL count: |erased symbols| + 2·|errors| ≤ parity    parity = RS(n,k) parity, recompute from Gate A trials
reference codec behaviour (illustrative parity budget; a char taints up to 2 bytes, so the UI must NOT promise a char count):
  erase within budget → recovered
  errors + erasures beyond budget → REFUSED (fail-closed)
```

Because one damaged character can taint two RS symbols, the real budget is *symbol* erasures after the bit-to-byte mapping, not characters. The UI must present the true symbol-level margin, never advertise "you may damage N characters."

base32 repack, footer marker, and checksum

base32	RS codeword bytes → custom-base32 symbols (5 bits/char; recompute counts from Gate A trials) alphabet = qpzry9x8gf2tvdw0s3jn54khce6mua7l (reuses the Bech32 charset, 5 bits/char)
marker	a STRUCTURAL marker the OCR locates – embedded so the footer reads like a URL path/query, NOT a conspicuous labelled token on the printed page
checksum	Bech32-derived polymod (BIP-173 generator constants) over the data symbols; NOT a standard Bech32 string (no "1" separator; >90 chars). Detects residual clean-capture typos the RS layer would otherwise spend parity on
footer	marker data checksum, shaped to resemble the page's source URL length: recompute

Three integrity layers, in decode order: the polymod checksum is a fast clean-capture pass/fail; Reed–Solomon is the error/erasure correction; and the final acceptance authority is the **AEAD tag** (encryption is mandatory), supplemented at the wallet level by the **derived-xpub-versus-independently-stored-descriptor** comparison before funding. A checksum failure must never discard an RS-recoverable footer — RS correction runs first except on the clean-capture fast path, and only a failure of the AEAD tag is fatal to the decode.

SELECTED DIRECTION • LOCAL STRUCTURAL EXTRACTION ONLY

The reader locates the structural marker inside the URL-like footer and reads the fixed-length field by position. It must scan the page **locally** — no URL is contacted, no network is reached — and the generated PDF/HTML is **self-contained** (no remote fonts, images, scripts, trackers, or active links). Camera use is **mode-locked**: this is the A1 footer-capture mode; the reader must never auto-detect a seed QR or accept arbitrary QR classes. The footer is self-describing and self-correcting; the surrounding recipe/tab/travel text is camouflage only and is not parsed.

Turning a photo of the footer into trustworthy capsule bytes

RULE	BEHAVIOUR
Confidence floor	a character is accepted only if its top-candidate OCR confidence $\geq$ a Gate-A floor (a candidate constant, not yet set); otherwise $\rightarrow$ erasure.
Top-two margin	even above the floor, if the margin to the second candidate $<$ a candidate (not-yet-set) threshold $\rightarrow$ erasure (ambiguous glyphs like the bech32-excluded 1/b/i/o never arise, but look-alikes still do).
Never guess	uncertainty always becomes an erasure (cost 1), never a low-confidence pick (a silent misread costs 2 and risks the tag).
Multi-frame	combine frames to lower the erasure count; a symbol confidently agreeing across frames is accepted, disagreement $\rightarrow$ erasure.
Bounded	hard cap on frames and total attempts per session; the whole OCR $\rightarrow$ RS $\rightarrow$ capsule chain runs behind one fail-closed exception boundary that zeroises on exit.
Gate-A constants	the floor, the margin, and the frame budget are intended to become reproducible-build-controlled constants (versioned and CI-checked) once fixed at Gate A — none are set yet.

Echo the recovered footer back to the user (grouped, see §06) so a stubborn misread can be hand-corrected — the same "echo the interpreted input for audit" discipline the Blueprint took from the Coleman tool, applied to OCR.

GATE A TESTS THE REAL DISGUISED PRINTED WEBPAGE — NOT A CLEAN LAB TOKEN

Gate A must run on the actual disguised-webpage design, not a pristine token in a lab. It must measure: **URL plausibility** of the footer (does it read as a believable source URL?), font size, page aging, folds, stains, photocopies, camera angle, first-pass OCR success, residual erasure distribution, false-accept rate, and the manual-fallback completion/error rates — and specifically whether **ordinary observers notice the footer as anomalous**. Only after this study may the encoded length,

RS(n,k), base32 counts, and any transcription grouping be recomputed and fixed; the capsule crypto (ChaCha20-Poly1305) is already frozen.

06 AGILITY & FALLBACK

Two version axes, and a fallback that keeps the capsule sealed

VERSION AXIS	FIELD	BUMPED WHEN...
Document coding	coding version marker	the RS params, alphabet, or framing change → next coding version. The reader dispatches on the marker, so old documents stay decodable.
Capsule crypto	capsule version field	the AEAD, key handling, or nonce/AAD rules change → next capsule version, bound into the AEAD AAD so a version swap fails authentication.

The two axes are orthogonal: the coding can evolve without touching the crypto and vice-versa. Both are checked before use; an unknown value on either axis is refused, not guessed.

OCR-FAILURE FALLBACK – A LAST RESORT, NOT "FOOLPROOF", AND NEEDS HUMAN TRIALS

When OCR + RS cannot recover (a badly damaged document), one fallback is to **manually transcribe the same footer characters** in numbered groups; the polymod checksum catches typos and RS fixes a few. But hand-typing that many characters is a *last-resort technical possibility, not a foolproof method*, and it risks breaking the URL-like disguise. Before assuming it is operationally acceptable, run **timed human trials** with damaged printouts, realistic fonts, lighting, and camera alignment, measuring first-pass OCR success, residual erasure distribution, frame count/RAM, false rejection, and manual completion/error rates across print aging, folds, stains, and low-resolution copies.

A separately printed, numbered human-transcription representation of the same payload may be worth more than raw characters — but the transcription grouping **must NOT be optimized while the payload is still unstable**. The frozen AEAD fixes the capsule crypto, but the Gate-A print/OCR trials still set the encoded length and any group layout; the physical grouping is not fixed to optimize yet. Only once the payload is stable should the transcription representation be designed and then **measured with real users**. It is a **policy decision**, not a silent addition, and it must not become a new plaintext-secret leak or a conspicuous crypto marker. Under the 2-of-3 wallet policy, possession of Seed A alone is not sufficient to spend (A1+B or A1+C is required); even so, any additional human-readable Seed-A representation must never appear as naked BIP-39 words or a labelled token on A1.

07 CROSS-CUTTING

The A2 decryption key, and camouflage as a system requirement

NO CIRCULAR DEPENDENCY — A2 LIVES ON BOTH B AND C

A1's footer ciphertext is decrypted under a per-document key **derived from A2 by HKDF-SHA256** (salt = wallet_id, info = "QuietKey/A1/v1"); A2 is a fresh 256-bit random root key generated at provisioning and **duplicated onto both cards B and C** (then wiped from the terminal). Because A2 is on both bearer cards, losing either card still leaves the other able to release A2 — there is no primary-card single point of failure. A2 is exportable from a bearer card by design; Seed B and Seed C are never exportable. The recovery paths (see the [Recovery Decision](#)) are **A1+B** and **A1+C**: a bearer card releases A2, the terminal decrypts A1, and A + the card sign.

What each object reveals alone

A1 alone reveals nothing — the footer is authenticated ciphertext and the page is ordinary camouflage.

Card C alone yields a signing key plus A2, but not A1, so it still cannot spend. **A1+B** and **A1+C** are the

B or C alone yields a signing key plus A2, but *not* A1, so it still cannot spend. **A1+B** and **A1+C** are the two decrypt-and-sign paths. If A1 is lost, **B+C** spends with A bypassed (2-of-3). Losing two signing factors can be unrecoverable — state that plainly. The terminal wipes A2 and Seed A after use, and the page is always scanned **locally** (no URL contacted, no network).

CAMOUFLAGE IS A SYSTEM REQUIREMENT, NEVER A CRYPTOGRAPHIC FACTOR

The disguise of A1 — an ordinary printed webpage whose footer resembles its source URL, safe among ordinary papers — is a first layer of defence and a product requirement across artifact, document format, UI, labels, and packaging. It is **never** a cryptographic factor and never a substitute for the 2-of-3 threshold and mandatory encryption. A naked BIP-39 phrase, a conspicuous `cv0 /token`, a crypto label, or an obvious recovery block on A1 defeats the product and is rejected.

RECOMPUTE THE PRINT-CODEC SIZES FROM GATE A TRIALS — THE CRYPTO CAPSULE IS FROZEN

The encoded length, $RS(n,k)$, base32 symbol counts, and any transcription grouping **must be recomputed from Gate A print/OCR trials**. The capsule crypto is frozen — ChaCha20-Poly1305 with `coding-ver || crypto-ver || nonce(12) || ciphertext(32) || tag(16)` and an HKDF-SHA256 document key — so the payload size no longer moves; only the physical print sizing does. Do not tune the old 49/83/133/142 figures — they belong to the rejected fixed-nonce capsule. Any size tuning also depends on Gate A's measured recovery success on the real disguised-webpage design (first-pass and completed recovery under damage, aging, and realistic capture), not on a stated preference for the custom codec.

Open items

- Independently implement the frozen ChaCha20-Poly1305 capsule (HKDF-SHA256 document key, stored 96-bit nonce, full-wallet_id AAD, 128-bit tag) twice and publish encode/decode/AEAD test vectors — then recompute $RS(n,k)$, base32 counts, and encoded length from the Gate A print/OCR trials.
- Fix the Gate-A reader constants (confidence floor, top-two margin, frame budget) from a real IMX708-class capture study on the disguised webpage — not yet set.

- Run the Gate-A disguised-webpage study (URL plausibility, font, aging, folds, stains, photocopies, angle, false-accept, manual fallback, observer-anomaly) before fixing the physical footer layout, group numbering, and the camera-aperture alignment.
- Confirm the wallet-level acceptance gate: derive Seed-A xpub and compare to the independently-stored descriptor, and require scan-back verification before funding.

QK2 Recovery Document (A1) · selected direction, not implemented · memo v7 · 2026-08-17

A1 is an ordinary printed webpage whose footer resembles its source URL and carries a MANDATORY authenticated encryption of the 256-bit Seed-A entropy, decryptable only with A2 (held on both cards B and C) plus the replaceable terminal. The RS/GF(256) codec, custom base32 repacking, and Bech32-derived polymod checksum were implemented in pure stdlib as a reference codec round-trip (coding layer only): clean recovery, erasure recovery to the RS parity budget, and fail-closed refusal beyond it — a level-1 reference computation, not an implementation test of the recovery design. Generator: `qk2_recdoc.py` . The AEAD is now fixed — **ChaCha20-Poly1305 (RFC 8439)** keyed by an HKDF-SHA256 document key (IKM=A2, salt=wallet_id) with a stored random 96-bit nonce and a 128-bit tag — though it is not computed in the stdlib reference codec; the old 49/83/133/142 figures assumed a rejected fixed-nonce 49-byte capsule and are not normative — the print-codec sizes must be recomputed from Gate A trials. Companion to the QuietKey QK2 Blueprint, QKEC-1 Specification, Standards Register, PSBT Checklist, Card Protocol, and the Recovery Decision.